

PySpark Cookbook für Microsoft Fabric

Best Practice Vorlagen & Cheat Sheet

Praktischer Leitfaden für Data Engineers: Produktionsreife Code-Snippets für Fabric Notebooks

Ingenieur-Prinzip: Man muss nicht hunderte PySpark-Methoden auswendig lernen. In 95% aller realen Enterprise-Aufgaben werden dieselben standardisierten Muster verwendet (Lesen $\rightarrow$ Bereinigen $\rightarrow$ Verknüpfen $\rightarrow$ Aggregieren $\rightarrow$ Optimierte Delta-Speicherung).

1. Basis-Imports (Immer in der ersten Zelle des Notebooks)

```
from pyspark.sql.functions import (
    col, lit, when, coalesce, concat, concat_ws,
    to_date, to_timestamp, year, month, dayofmonth, date_format,
    round, sum, avg, count, min, max, row_number
)
from pyspark.sql.window import Window
```

2. Daten einlesen (Data Ingestion)

```
# 1. Delta-Tabelle aus dem aktuellen oder verknüpften Lakehouse Lesen
df = spark.read.table("lh_bronze.dbo.publicholidays")

# 2. Parquet-Datei aus dem OneLake Files/ Verzeichnis Lesen
df_parquet = spark.read.parquet("Files/sample_datasets/public_holidays.parquet")

# 3. CSV-Datei mit automatischer Schema-Erkennung und Kopfzeile einlesen
df_csv = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("Files/sample_datasets/city_safety_seattle.csv")
```

3. Datenbereinigung, Typkonvertierung & NULL-Handling

```
# Zeilen filtern und Spalten umbenennen
df_clean = df \
    .filter(col("date") >= "2020-01-01") \
    .filter(col("holidayName").isNotNull()) \
    .withColumnRenamed("countryRegionCode", "CountryCode")

# String in Datum konvertieren und Zeitattribute extrahieren
df_dates = df_clean \
    .withColumn("HolidayDate", to_date(col("date"))) \
    .withColumn("Year", year(col("HolidayDate"))) \
    .withColumn("DayName", date_format(col("HolidayDate"), "EEEE"))

# NULL-Werte ersetzen (coalesce) und bedingte Logik (when / otherwise)
df_final = df_dates \
    .withColumn("IsPaid", coalesce(col("isPaidTimeOff"), lit(False))) \
    .withColumn("Priority",
        when(col("DayName").isin("Saturday", "Sunday"), "Weekend")
        .otherwise("Workday")
    )
```

4. Tabellen verknüpfen (Joins)

```
# Innerer Join (INNER JOIN) über gemeinsamen Schlüssel
df_joined = df_orders.join(df_customers, on="CustomerID", how="inner")

# Linker Join (LEFT JOIN) bei unterschiedlichen Schlüsselnamen
df_enriched = df_sales.join(
    df_products,
    df_sales["ProdKey"] == df_products["ProductID"],
    how="left"
).drop(df_products["ProductID"]) # redundante Schlüsselspalte entfernen
```

5. Aggregationen & Gruppierungen (Group By)

```
# Zusammenfassung: Kennzahlen nach Land und Jahr
df_summary = df_final.groupBy("CountryName", "Year") \
    .agg(
        count("HolidayName").alias("TotalHolidays"),
        sum(when(col("IsPaid") == True, 1).otherwise(0)).alias("PaidHolidays")
    ) \
    .orderBy(col("Year").desc(), col("TotalHolidays").desc())
```

6. Analytische Fensterfunktionen (Window Functions)

```
# Zeilen-Rangfolge: jeweils jüngstes Ereignis pro Partition ermitteln
window_spec = Window.partitionBy("CountryName").orderBy(col("HolidayDate").desc())

df_ranked = df_final \
    .withColumn("RankNum", row_number().over(window_spec)) \
    .filter(col("RankNum") == 1) # nur den jeweils aktuellsten Datensatz behalten
```

7. In Delta Lake speichern (Silver & Gold)

```
# Tabelle komplett überschreiben (Overwrite) – für Dimensionen und Stammdaten
df_final.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("lh_silver.dbo.dim_holidays")

# Neue Datensätze anhängen (Append) – für Fakten und Transaktionsdaten
df_new_records.write \
    .format("delta") \
    .mode("append") \
    .saveAsTable("lh_gold.dbo.fact_daily_events")
```

8. Delta Lake Wartung & Time Travel (%%sql)

```
-- 1. Dateikomprimierung und Z-Order Clustering für maximale Abfrage-Geschwindigkeit
OPTIMIZE lh_silver.dbo.dim_holidays ZORDER BY (HolidayDate);

-- 2. Transaktionslog und Versionshistorie prüfen
DESCRIBE HISTORY lh_silver.dbo.dim_holidays;

-- 3. Time Travel: Historischen Zustand zu einer bestimmten Version abfragen
SELECT * FROM lh_silver.dbo.dim_holidays VERSION AS OF 0;

-- 4. Bereinigung veralteter Datendateien (Standard: älter als 7 Tage)
VACUUM lh_silver.dbo.dim_holidays;
```

Fabric Best Practice: Die `V-Order` Optimierung ist in Microsoft Fabric bei jedem Schreibvorgang via `saveAsTable()` standardmäßig aktiviert. Sie sortiert und komprimiert Parquet-Dateien so, dass Power BI im Direct Lake Modus und SQL-Endpunkte Abfragen in Millisekunden beantworten.